

Hashing in FinTech: 8-Level Hackathon

A comprehensive, language-neutral algorithmic challenge spanning Payment Gateways, Fraud Detection, Trading Platforms, and Distributed Ledgers — from Intermediate to Expert difficulty.

DEVELOPED BY TALENCIAGLOBAL

LANGUAGE NEUTRAL · JAVA · PYTHON · C++

Hackathon Overview

Challenge at a Glance

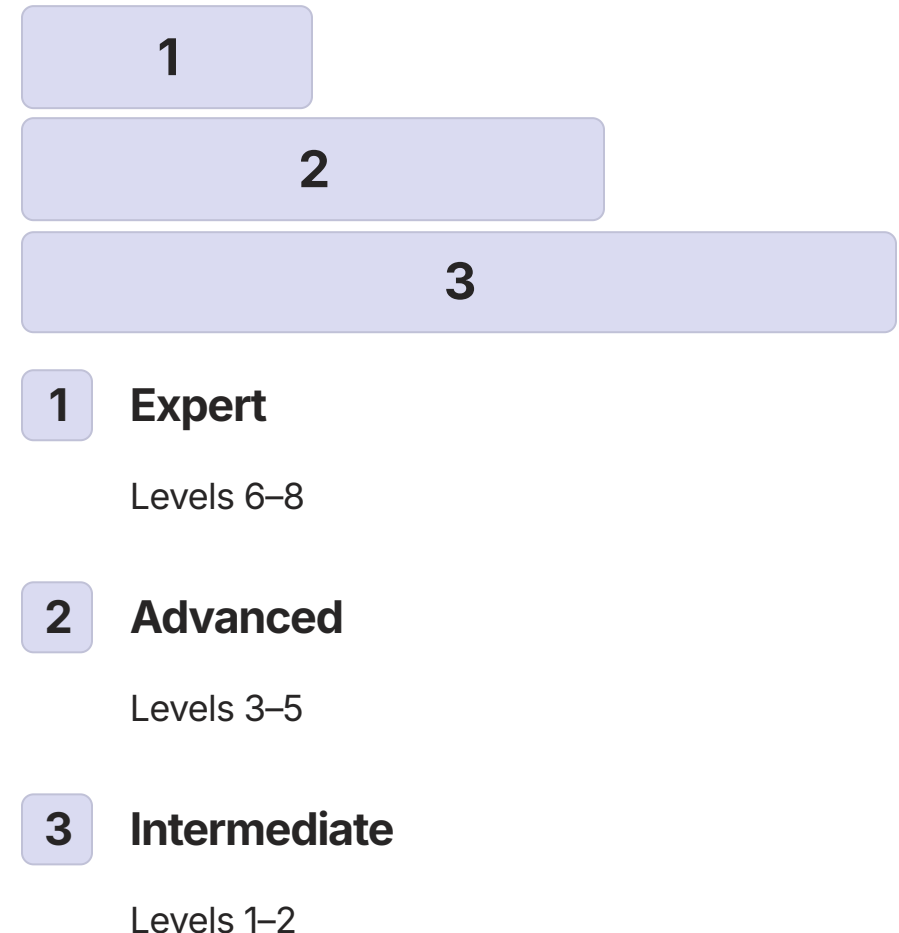
Theme: Building a Complete Real-time FinTech Processing System using Hashing data structures

Domain: FinTech — Payments, Fraud Detection, Trading, Banking, Compliance

Language: Language Neutral — Pseudo-code / Algorithmic specifications applicable to Java, Python, and C++

Prerequisites: Hashing concepts, hash tables, collision handling, $O(1)$ complexity analysis

Difficulty Progression



i Industry context: Global digital payment transaction volume exceeded **\$8.49 trillion in 2023**, with fraud losses surpassing **\$485 billion** — making high-performance hashing infrastructure mission-critical for every FinTech platform.

Level 1: Core HashMap Implementation

PAYMENT GATEWAY

INTERMEDIATE

Implement a high-performance hash map from scratch for a payment gateway processing **millions of transactions daily**. Your hash map must handle collisions using separate chaining with linked lists — the foundational data structure underpinning every modern payment processor.

PaymentHashMap Requirements

- Separate chaining with linked lists per bucket
- Initial capacity: 16 buckets
- Load factor threshold: 0.75
- Dynamic resizing — double capacity when threshold exceeded
- All operations $O(1)$ average, $O(n)$ worst-case

Core Methods

- `put(key, value)` — Insert or update key-value pair
- `get(key)` — Return value or null if not found
- `remove(key)` — Remove and return value
- `containsKey(key)` — Boolean existence check
- `size()`, `isEmpty()`, `clear()`

Financial Hash Functions

UPI ID Hash

Format:
username@bank —
e.g., raj@okhdfc

Account Number Hash

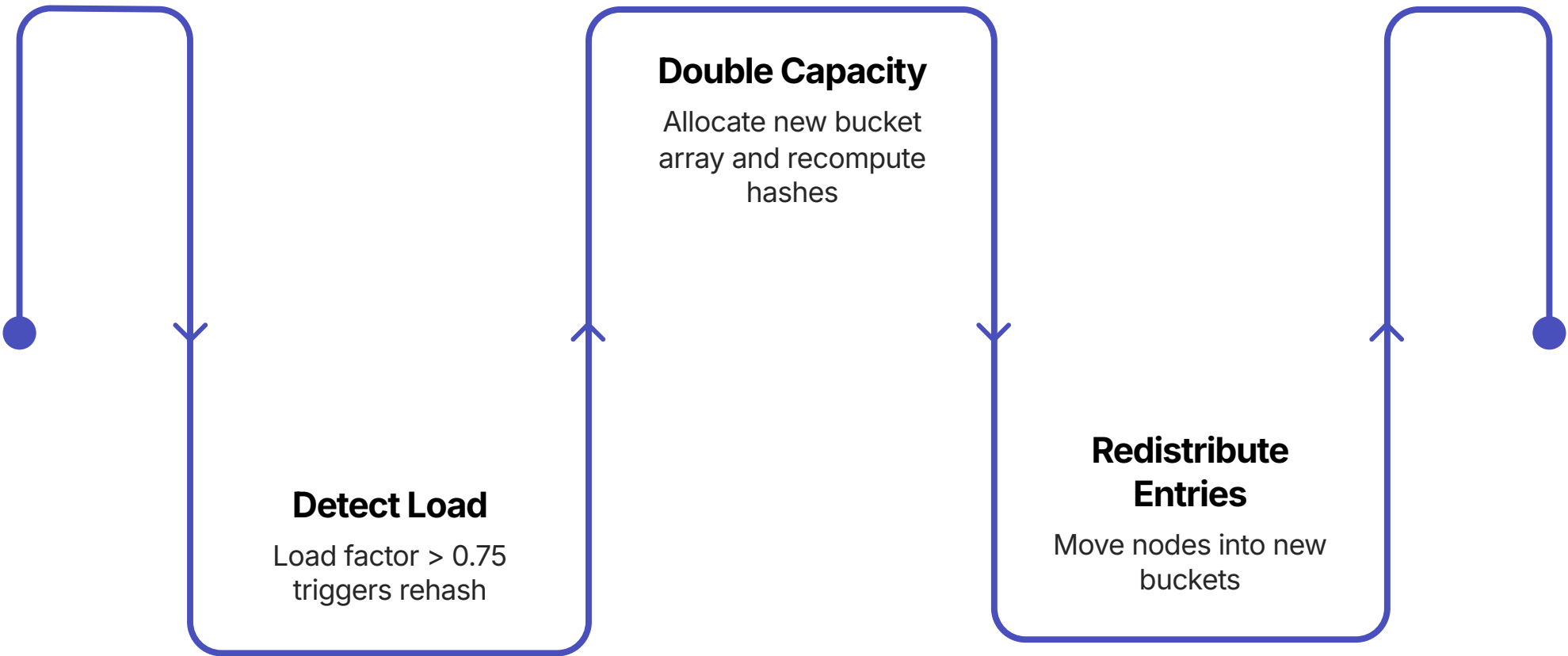
16-digit string with
avalanche effect

Transaction ID Hash

Format:
TXN_YYYYMMDD_seque
ntial

IFSC Code Hash

11-character
alphanumeric identifier



Rehashing is triggered automatically when the load factor crosses 0.75. After 12 inserts into a capacity-16 map, the load factor reaches exactly 0.75, triggering expansion to 32 buckets with full redistribution.

✅ **Acceptance Criteria:** 1,000,000 inserts must complete in under 2 seconds. All hash functions must pass uniform distribution tests with collision variance below 5%.

Level 2: Fraud Detection Engine

HASHSET & FREQUENCY MAPS

INTERMEDIATE

Build a real-time fraud detection system using hash sets and frequency maps to identify duplicate transactions, rate limit violations, and suspicious behavioral patterns. According to the RBI Annual Report 2023, **over ₹2,537 crore** was lost to digital payment fraud in India alone — making real-time detection indispensable.

Duplicate Detector

Hash set with 5-second sliding window. Returns `isDuplicate()` in $O(1)$. Auto-cleanup of expired entries.

Rate Limiter

HashMap: `userID` $\rightarrow$ timestamp queue. Enforces RBI mandate of ~17 transactions/minute (system cap: 20/min).

Money Mule Detector

Flags users sending to **>50 unique recipients** in 1 hour — a primary indicator of money laundering networks.

Structuring Detector

Identifies 10+ transactions just below ₹50,000 in 30 minutes — a classic smurfing/structuring pattern.

FraudAlert Severity Levels

Alert Type	Severity	Trigger
DUPLICATE	MEDIUM	Same TXN ID within 5s
RATE_LIMIT	HIGH	21st transaction/minute
MONEY_MULE	HIGH	51+ unique recipients/hour
STRUCTURING	CRITICAL	10+ txns below ₹50K in 30min

Key Performance Targets

5s

Duplicate Window

Sliding time window for deduplication cache

20

Max TXN/Min

Per-user rate limit per RBI guidelines

<1%

False Positive Rate

Target accuracy for fraud pattern detection

Level 3: Order Book for Trading Platform

HASHMAP WITH SORTED KEYS

ADVANCED

Implement a real-time order book for a cryptocurrency exchange using hash maps that support range queries and sorted order for price levels. Global crypto exchange order books process upwards of **100,000 orders per second** at peak — demanding sub-millisecond data structure performance.

Order Book Architecture

BID Side
Buy orders — highest price retrieved first. O(1) lookup per price level.

ASK Side
Sell orders — lowest price retrieved first. O(log n) best ask retrieval.

OrderBook Methods

- addOrder(order) — Add to bid or ask side
- removeOrder(orderId) — Partial or full removal
- getBestBid() — Highest buy price
- getBestAsk() — Lowest sell price
- getPriceLevelsInRange(min, max, side)
- getMarketDepth(levels) — Volume per level

Sample Order Book State

Side	Price	Volume	Status
BUY	50,100	2.0	OPEN
BUY	50,000	0.5	PARTIAL
SELL	50,200	1.0	OPEN
SELL	50,050	0.0	FILLED

Matching Engine Logic

When SELL Order4 arrives at price 50,050, it matches against BUY Order1 at 50,000. Trade executes at **price=50,000, quantity=1.0** using price-time priority (FIFO within each price level). Spread = Best Ask – Best Bid = **100 units**.

☐ Acceptance: O(1) price level lookup, 100K orders/second matching throughput, accurate spread calculation at all times.

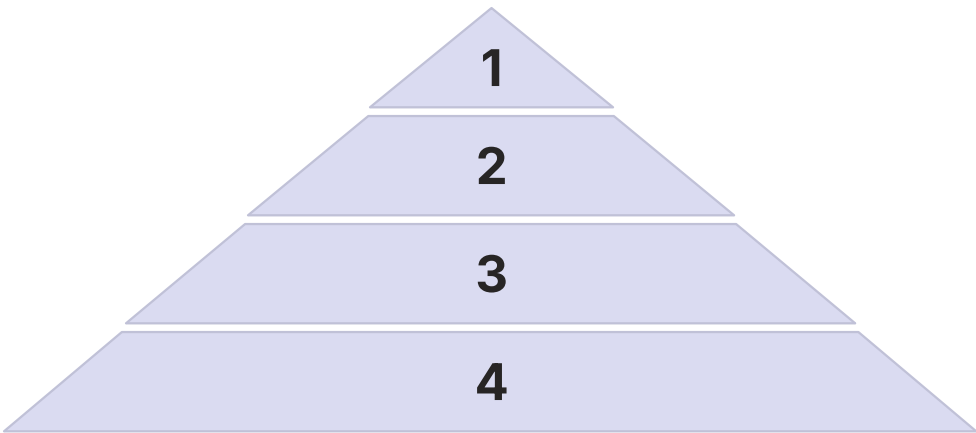
Level 4: Frequent Flyer Points System

HASHMAP + HEAP

ADVANCED

Build a loyalty points system for a payments app that tracks user transactions and identifies top spenders using a hash map combined with a min-heap for efficient Top-K queries. India's UPI ecosystem processed over **117 billion transactions worth ₹182 trillion in FY2024** — loyalty systems must scale to hundreds of millions of users.

Tier Thresholds



- 1

Platinum
100,000+ pts
- 2

Gold
25,000+ pts
- 3

Silver
5,000+ pts
- 4

Bronze
0+ pts

LoyaltyPointsTracker Design

- `addPoints(userId, points)` — $O(\log n)$ due to heap update
- `getUserPoints(userId)` — $O(1)$ via hash map
- `getTopUsers(k)` — $O(k \log n)$ via min-heap
- `getUserRank(userId)` — Rank 1 = highest points

Campaign Manager

- Hash set tracks which users received each campaign bonus
- Welcome bonus: 500 points, granted exactly once per user
- `grantCampaignBonus(userId, campaignId)` — idempotent
- Duplicate bonus prevention via $O(1)$ hash set lookup

✔ Sample: User C with 25,000 pts → Platinum. User D crosses 26,000 → Tier upgrade notification triggered automatically.

$O(1)$

`getUserPoints`

Direct hash map lookup, no traversal

$O(\log n)$

`addPoints`

Heap update on every points change

$O(k \log n)$

`getTopUsers(k)`

Min-heap extraction for top-K results

1x

Bonus Grant

Campaign bonuses issued exactly once per user per campaign

Level 5: Real-Time Fraud Rules Engine

HASHMAP OF PREDICATES

ADVANCED

Build a dynamic fraud detection rules engine where rules are stored in hash maps for O(1) lookup and can be added or removed at runtime without restarting the system. Enterprise fraud engines at major banks evaluate **over 10,000 rules per transaction** in under 1 millisecond — requiring highly optimized hash-based rule repositories.



Rule Repository

HashMap: ruleId → Rule. Supports dynamic add/remove, category grouping (AMOUNT, VELOCITY, LOCATION, DEVICE, TIME), and enable/disable without deletion.



Rule Evaluation Engine

Evaluates all enabled rules per transaction. Computes aggregate risk score (sum of violated rule weights). Blocks transaction if score exceeds configurable threshold.



Dynamic Parameter Store

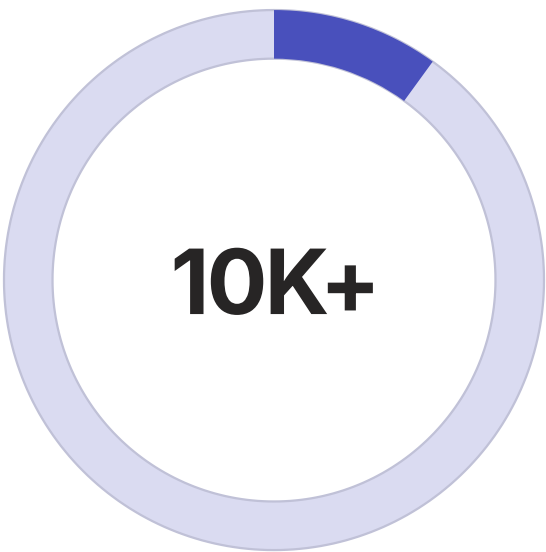
Nested HashMap: ruleId → paramName → value. Parameters updated live — e.g., changing amount threshold from ₹1,00,000 to ₹2,00,000 takes effect immediately without restart.

Sample Rule Evaluation

Rule ID	Condition	Weight	Action
R001	Amount > ₹1,00,000	50	FLAG
R002	Velocity > 20/min	80	BLOCK
R003	New device + Large amount	70	FLAG
R004	Suspicious location	90	FLAG

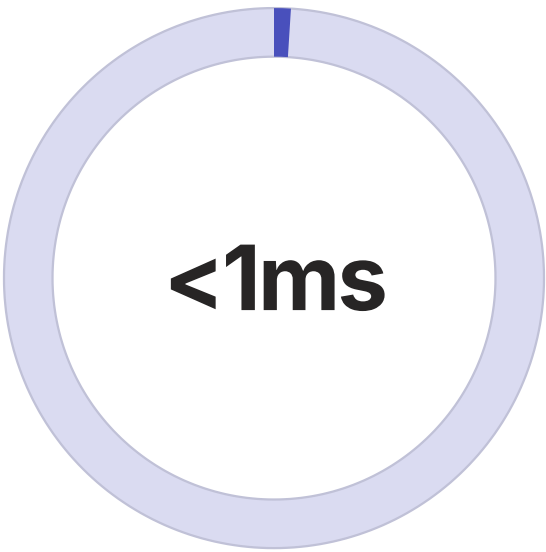
TXN002 with amount ₹2,00,000 and velocity 25/min violates R001 + R002 → Risk score: **130** → **BLOCKED** (threshold = 100).

Performance Target



Rules Supported

Sub-millisecond evaluation across full rule set



Evaluation Time

Per transaction across all active rules

Level 6: Merchant Payment Routing

CONSISTENT HASHING

EXPERT

Implement consistent hashing for a payment gateway to distribute transactions across multiple payment processors with minimal re-routing when processors are added or removed. Industry benchmarks show that naive modular hashing causes **~100% key remapping** on node changes, while consistent hashing limits remapping to just **1/N of keys** — a critical advantage for zero-downtime payment infrastructure.

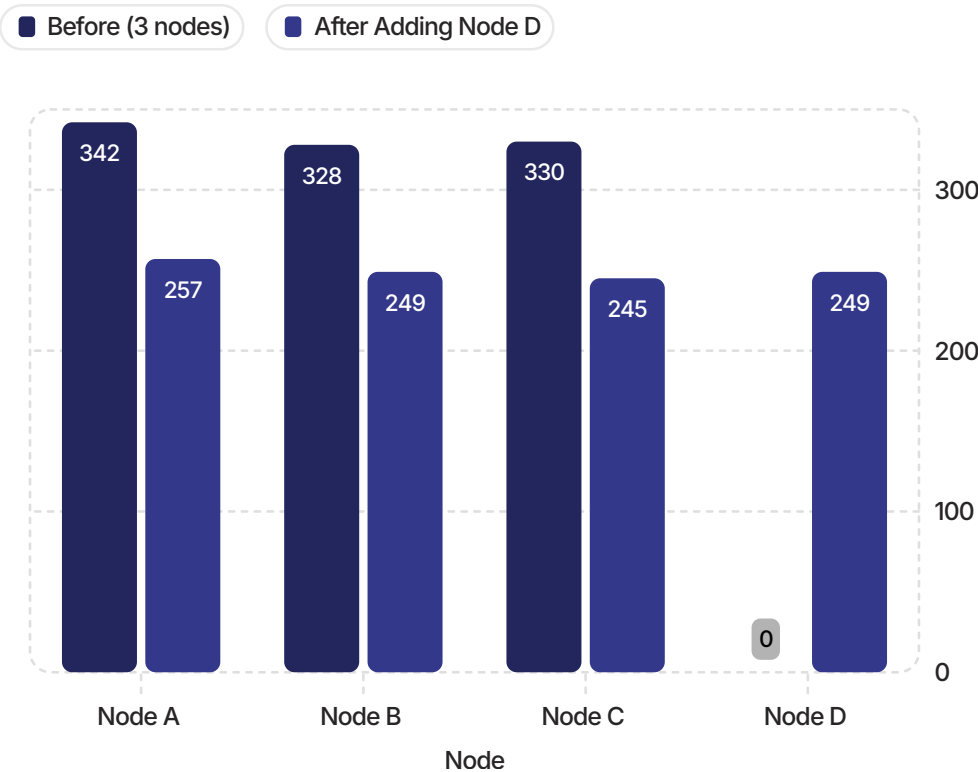
Consistent Hash Ring Design

- Virtual nodes: each physical node mapped to **100–200 virtual nodes**
- Hash ring: sorted map of hash value → physical node
- `addNode(nodeId, virtualNodeCount)` — minimal redistribution
- `removeNode(nodeId)` — only affected keys remapped
- `getNode(key)` — $O(\log n)$ clockwise lookup on ring

Payment Router Features

- Node health checking — skip unhealthy processors
- Automatic failover to next node on ring
- `markUnhealthy(nodeId)` / `markHealthy(nodeId)`

Distribution: 1,000 Transactions Across 3 Nodes



Adding Node D remaps only **24.9%** of keys — approximately 1/4, consistent with the 1/N theoretical guarantee. Distribution variance remains below 2%.

Acceptance Criteria: Adding a node redistributes only ~1/N of keys. Removing a node redistributes only keys from that node. Supports 100+ physical nodes and 10,000+ virtual nodes.

Level 7: Bloom Filter for Fraud Pattern Cache

PROBABILISTIC HASHING

EXPERT

Implement a Bloom filter for a fraud detection system that efficiently checks if a transaction pattern has been seen before, with configurable false positive rate but **zero false negatives**. Bloom filters are used by Google, Apache Cassandra, and major payment networks to eliminate unnecessary database lookups — delivering up to **100× speedup** for "definitely not fraud" determinations.

Bloom Filter Mathematics

Given expected insertions **n = 10,000** and desired false positive probability **p = 0.01**:

$$m = -\frac{n \ln p}{(\ln 2)^2} \approx 95,851 \text{ bits} \approx 12\text{KB}$$
$$k = \frac{m}{n} \ln 2 \approx 7 \text{ hash functions}$$

Double hashing technique:
 $h_i(x) = (h_1(x) + i \cdot h_2(x)) \bmod m$ generates k independent hash values efficiently from just two base hash functions.

Fraud Pattern Cache Performance

Metric	Value
Total queries	10,000
Bloom hits (potential fraud)	500
Bloom misses (definite not-fraud)	9,500
False positives	3 (0.03%)
False negatives	0 (guaranteed)
DB lookups avoided	9,500 (95%)

Scalable Bloom Filter

When the primary filter reaches capacity, a new filter is chained. After 10,000 inserts, 2 filters exist in chain with total capacity 20,000 and current false positive rate of **0.5%** — well within the 1% target.

Zero False Negatives

A known fraud pattern will *a/ways* be flagged — the core safety guarantee of Bloom filters.

Configurable FP Rate

Trade memory for accuracy. 1% FP at 12KB; 0.1% FP at ~19KB for same 10K elements.

Unlimited Scale

Scalable Bloom Filter chains new filters on saturation — supports unlimited insertions with bounded FP rate.

100× DB Speedup

Bloom misses bypass database entirely — critical for sub-millisecond fraud screening at scale.

Level 8: Distributed Payment Ledger

CONSISTENT HASHING + MERKLE TREE

EXPERT · FINAL INTEGRATION

Build a distributed payment ledger using consistent hashing for sharding and Merkle tree hashing for integrity verification across millions of transactions. Blockchain-based payment ledgers now secure over **\$2.5 trillion in annual transaction value**, with Merkle proofs enabling cryptographic verification without downloading the full dataset.

Sharded Ledger: 4-Node Distribution

Shard	Hash Range	Node	Transactions
Shard 0	0 – 16,383	Node A	249,847
Shard 1	16,384 – 32,767	Node B	250,123
Shard 2	32,768 – 49,151	Node C	250,091
Shard 3	49,152 – 65,535	Node D	249,939

1,000,000 transactions distributed with **<0.05% variance** across shards — demonstrating consistent hashing's near-perfect uniformity at scale.

Merkle Tree Integrity

- **Leaf Hashing**
Each transaction hashed individually: `hash(TXN_data)`
- **Internal Nodes**
Parent = `hash(leftChild + rightChild)` up the tree
- **Root Hash**
Single fingerprint representing all transactions in shard
- **Global Root**
`hash(shard0Root + shard1Root + shard2Root + shard3Root)`

 **Tamper Detection:** Modifying any single transaction in Shard 1 causes that shard's Merkle root to change, which cascades to change the global root hash — making tampering immediately detectable with $O(\log n)$ proof verification.

Level 8 Bonus: Cryptographically Secure Hashing

BLOCKCHAIN + PROOF-OF-WORK

EXPERT · BONUS

Enhance the distributed ledger with SHA-256 cryptographic hashing and implement proof-of-work for transaction blocks — the same mechanism securing Bitcoin's \$1.3 trillion market cap. Each block must reference the previous block's hash, creating a tamper-evident chain where altering any block invalidates all subsequent blocks.

Block Structure

Block Header

blockId,
previousBlockHash,
timestamp, nonce,
merkleRoot

Block Body

List of transactions.
Merkle root computed
from all transaction
hashes.

Proof-of-Work

mineBlock(difficulty) —
iterate nonce until hash
has required leading zeros
(e.g., 0x0000... for
difficulty=4).

Chain Validation

verifyChain() — each
block's
previousBlockHash must
match prior block's
computed hash.

Sample Blockchain State

Block	Nonce	Hash Prefix
Genesis (0)	0	0x0000a3f2...
Block 1	152,364	0x0000c7e2.. .
Block 2	894,231	0x0000b9a1.. .

Block 1 required **152,364 nonce iterations** to find a hash with 4 leading zeros — demonstrating the computational cost that makes tampering economically infeasible.

 Chain verification: VALID. Block count: 3. Total transactions: 5,000. Tampering Block 1 invalidates Block 2's previousBlockHash reference immediately.

Hackathon Deliverables & Evaluation

Per-Level Deliverables (Levels 1–7)

01

Complete Implementation

All data structures and methods fully implemented per specification

02

Unit Tests

Minimum 10 unit tests per level covering edge cases and performance

03

Complexity Analysis

Big-O documentation for every operation — average and worst case

04

Design Documentation

Collision strategy, hash function choice, and architectural decisions

Level 8 Final Integration Deliverables

Performance Benchmarks

Insertion throughput (TXN/sec), query latency (p50/p95/p99), shard distribution uniformity, rebalancing time

Integrity Verification Demo

Live demonstration of Merkle proof generation and tamper detection

Audit Proof System

End-to-end audit proof generation and independent verification

Stress Test

Full system validation with 10 million transactions across all shards

Successful Completion Criteria

Each level has a precise, measurable acceptance benchmark. These performance targets reflect real-world production standards used by leading FinTech companies including Razorpay, PhonePe, and Stripe — where sub-second latency and correctness are non-negotiable.



Level 1

1M HashMap operations in **<2 seconds**. Chaining handles all collisions correctly.



Level 2

Fraud detection catches all patterns with **<1% false positive rate**.



Level 3

Order book matches **100K orders/second** with price-time priority.



Level 4

Top-K queries return correct results in **O(k log n)** time.



Level 5

Rules engine evaluates **10,000 rules in <1ms** per transaction.



Level 6

Consistent hashing redistributes **<25% of keys** on any node change.



Level 7

Bloom filter achieves **0% false negatives** and **<1% false positives**.



Level 8

Distributed ledger handles **10M transactions** with full integrity verification.

All hash functions must produce uniform distribution with variance below 5%. No collision chain may cause measurable performance degradation under load. **Good luck — may your hash functions be collision-free!** 🗝️💳🏦